

Decision Tree

Example 1

Gender	Occupation	Suggested
F	Student	PUBG
F	Programmer	Github
M	Programmer	WhatsApp
F	Programmer	Github
M	Student	PUBG
M	Student	PUBG

Decision Tree = Nested if else

```
if occupation == student:  
    print (PUBG)  
else
```

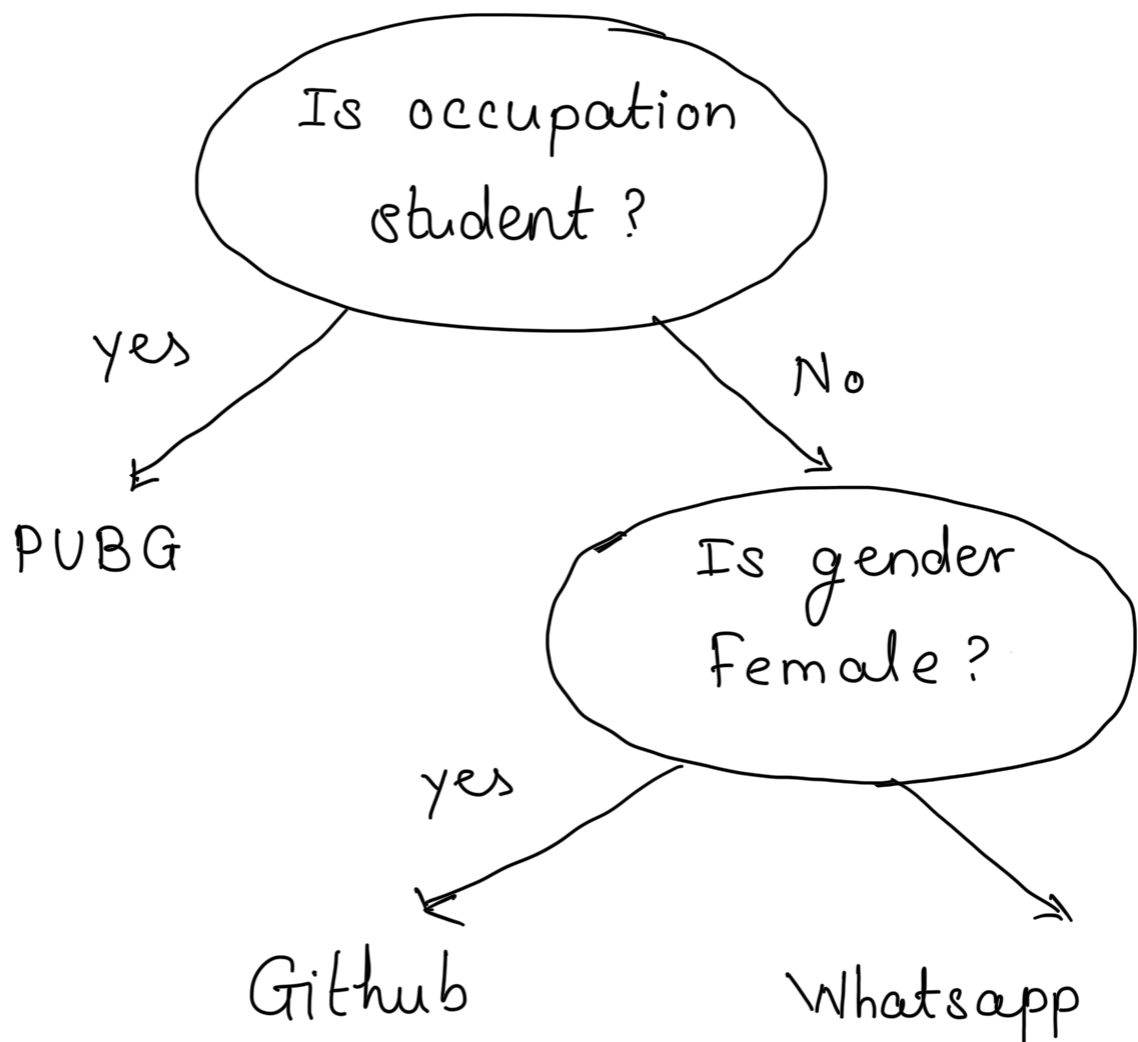
If gender == Female

print (Github)

else

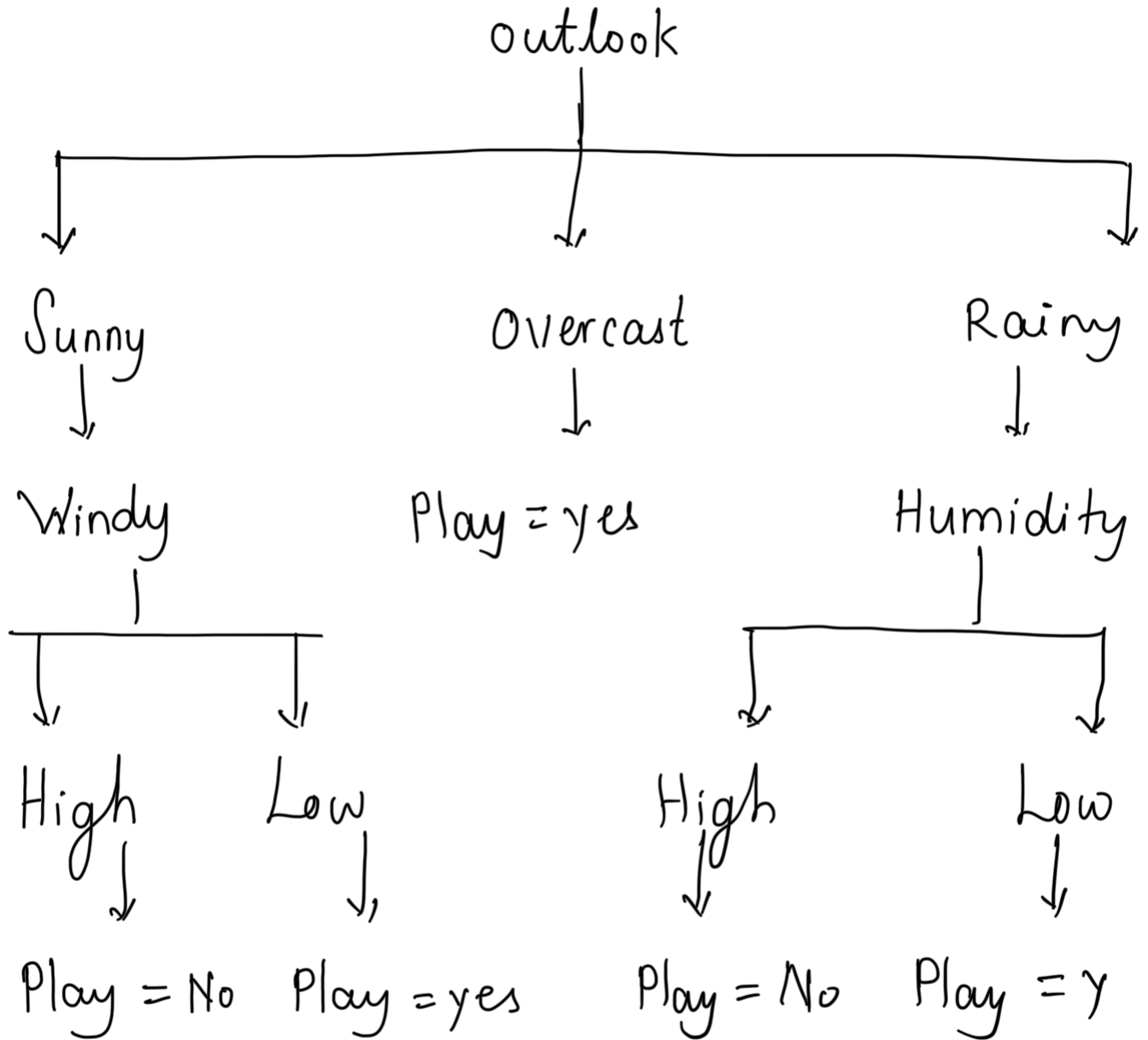
print (WhatsApp)

Where is the Tree?



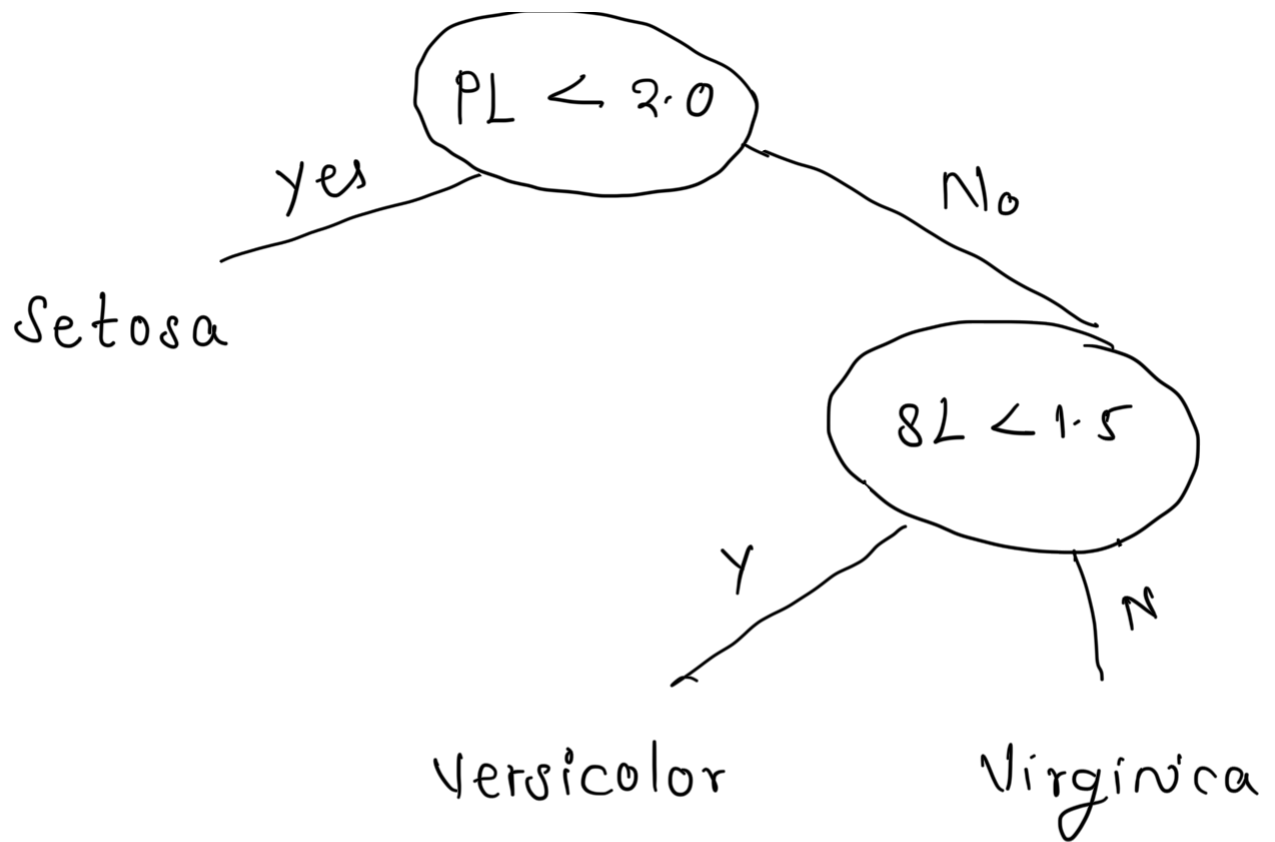
Example 2

Day	Outlook	Temp	Humid	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

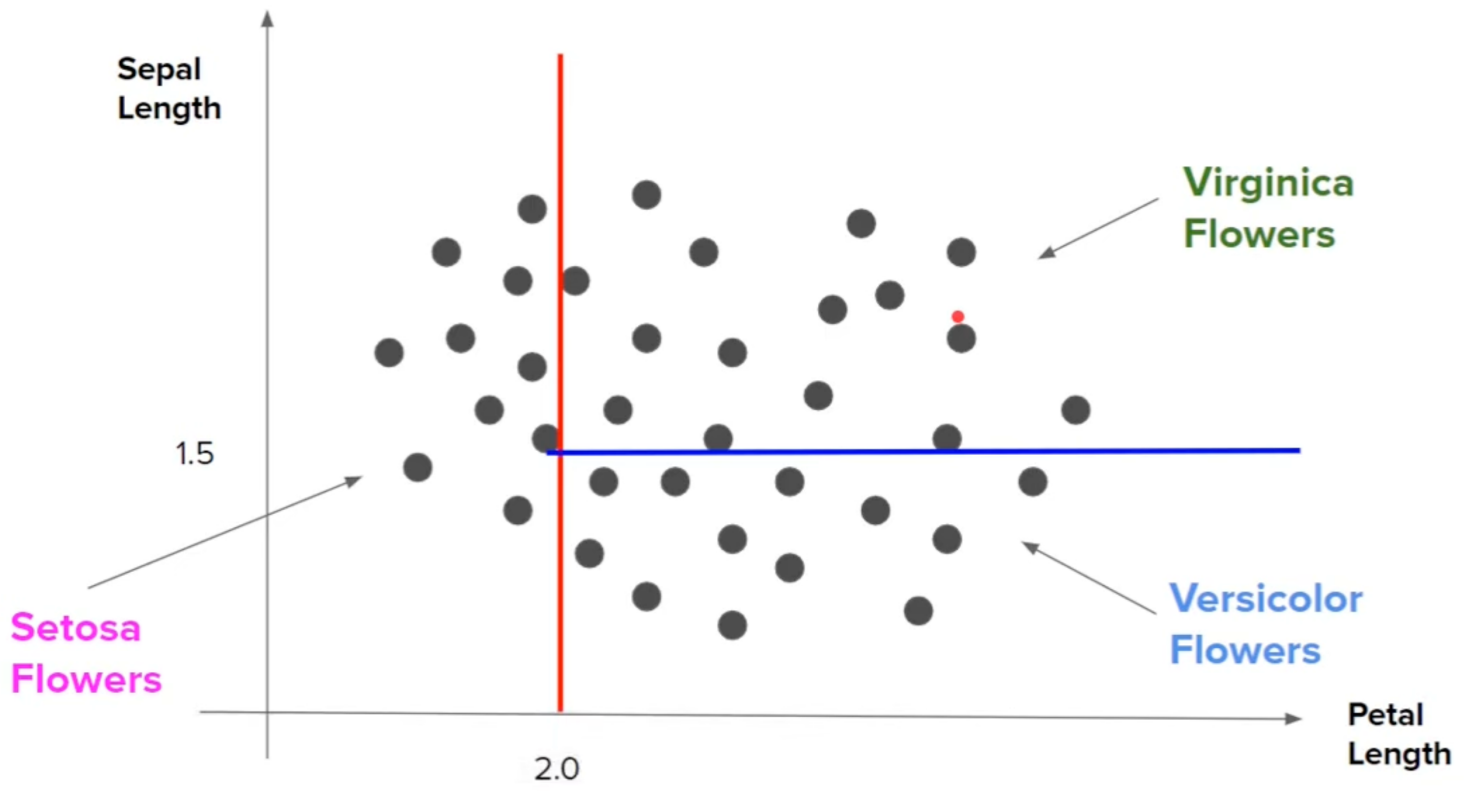


What if we have numerical data?

Petal Length	Sepal Length	Type
1.34	0.34	Setosa
3.45	1.45	Versicolor
1.69	0.98	Setosa
2.56	1.79	Virginica
3.00	1.13	Versicolor
1.3	0.88	Setosa



Geometric Intuitions



Pseudo Code

1) Begin with your training dataset, which should have some feature variables & classification or regression output.

2) Determine the "best Feature" in the dataset to split the data on; more on how we define "best Feature" later.

3) Split the data into subsets that contain the correct values for this best Feature. This splitting basically defines a node on the tree i.e. each node is a splitting point based on a certain Feature from our data.

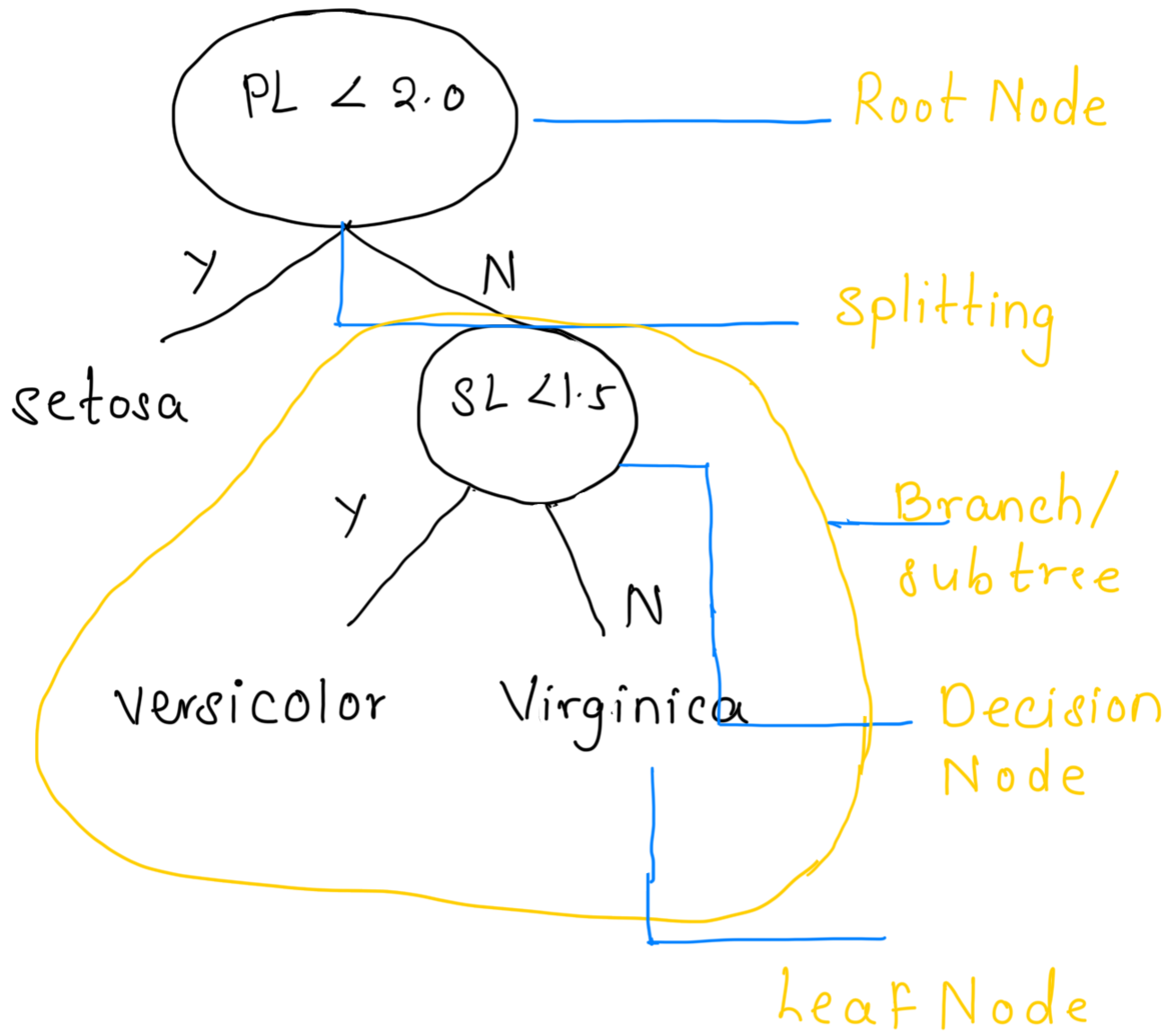
4) Recursively generate new tree nodes by using the subset of data created from step 3.

Conclusion

Programatically speaking - Decision trees are nothing but a giant structure of nested if-else condition.

Mathematically speaking, Decision trees use hyperplanes which run parallel to any one of the axes to cut the coordinate system into hyper cuboids.

Terminology



Advantages

1. Intuitive and easy to understand
2. Minimal data preparation is required

3. The cost of using the tree for inference is logarithmic in the number of data points used to train the tree.

Disadvantage

1) Overfitting

2) Prone to errors for imbalanced datasets

CART - Classification And Regression Trees

The logic of decision trees can also be applied to regression problems, hence the name CART.

Decision Tree Entropy

What is Entropy

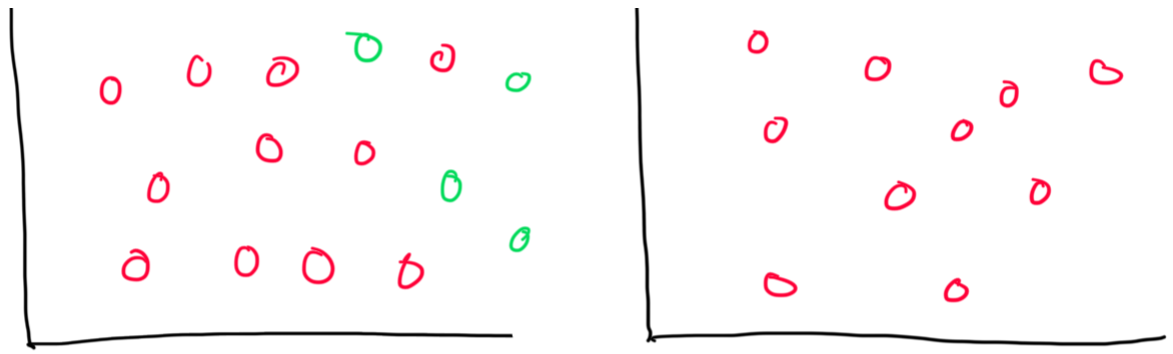
In most layman terms, Entropy is nothing but the measure of disorder, or we can call it as measure of purity / impurity
example

Ice $\Rightarrow$ tightly packed molecules, can't move

Water $\Rightarrow$ Can move to some extent

Vapor $\Rightarrow$ molecules can move everywhere as they are not closely packed.





More knowledge less entropy

Graph 1 has more entropy

How to calculate Entropy

The mathematical formula for entropy is

$$E(S) = \sum_{i=1}^c -P_i \log_2 P_i$$

Where,

P_i = frequentist probability of an element/class "i" in our data.

For eg. if our data has only 2 class labels **Yes** and **No**

$$E(D) = -P_{yes} \log_2(P_{yes}) - P_{no} \log_2(P_{no})$$

Salary	Age	Purchase
20000	21	Y
10000	45	N
60000	27	Y
15000	31	N
12000	18	N

$$H(D_1) = -P_Y \log_2(P_Y) - P_N \log_2(P_N)$$

$$= -\frac{2}{5} \log_2(2/5) - \frac{3}{5} \log_2(3/5)$$

$$H(D_1) = 0.97$$

Salary	Age	Purchase
34000	31	N
15000	25	N
69000	57	Y
25000	21	N
32000	28	N

$$H(d_2) = -P_Y \log_2(P_Y) - P_N \log_2(P_N)$$

$$= \frac{-1}{5} \log_2(1/5) - \frac{4}{5} \log_2(4/5)$$

$$H(d_2) = 0.72$$

Entropy of $d_1 > d_2$

Salary	Age	Purchase
--------	-----	----------

20000	21	N
10000	45	N
60000	27	N
15000	31	N
12000	18	N

$$H(d_3) = -P_y \log_2(P_y) - P_n \log_2(P_n)$$

$$= -\frac{0}{5} \log_2(0) - \frac{5}{5} \log_2(5/5)$$

$$H(d_3) = 0$$

Calculating entropy for a 3 class problem

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes

15000	31	No
30000	30	Maybe
12000	18	No
40000	40	Maybe
20000	20	Maybe

$$H(d) = -P_y \log_2(P_y) - P_n \log_2(P_n) - P_m \log_2(P_m)$$

$$= -\frac{2}{8} \log_2(2/8) - \frac{3}{8} \log_2(3/8)$$

$$- \frac{3}{8} \log_2(3/8)$$

$$H(d) = 1.56$$

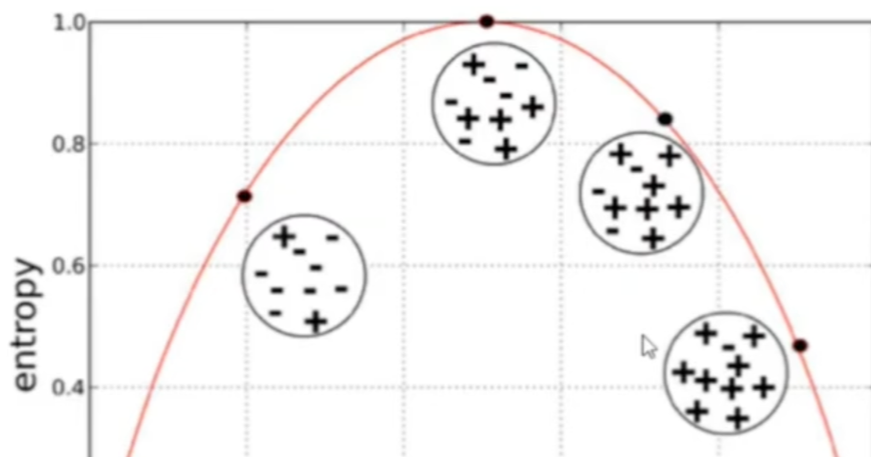
Observation

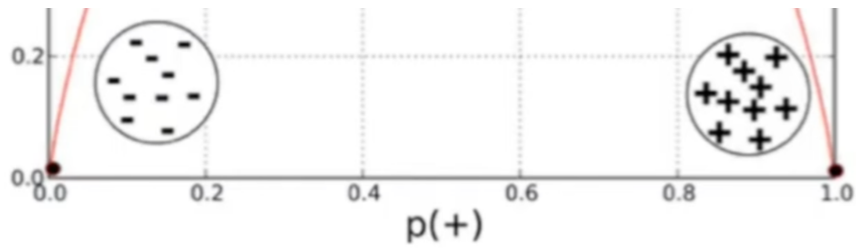
- More the uncertainty more is entropy

1.5

- For a 2 class problem the min entropy is 0 & the max is 1
- For more than 2 classes the min entropy is 0 but the max can be greater than 1
- Both $\log_2$ or $\log_e$ can be used to calculate entropy.

Entropy Vs Probability

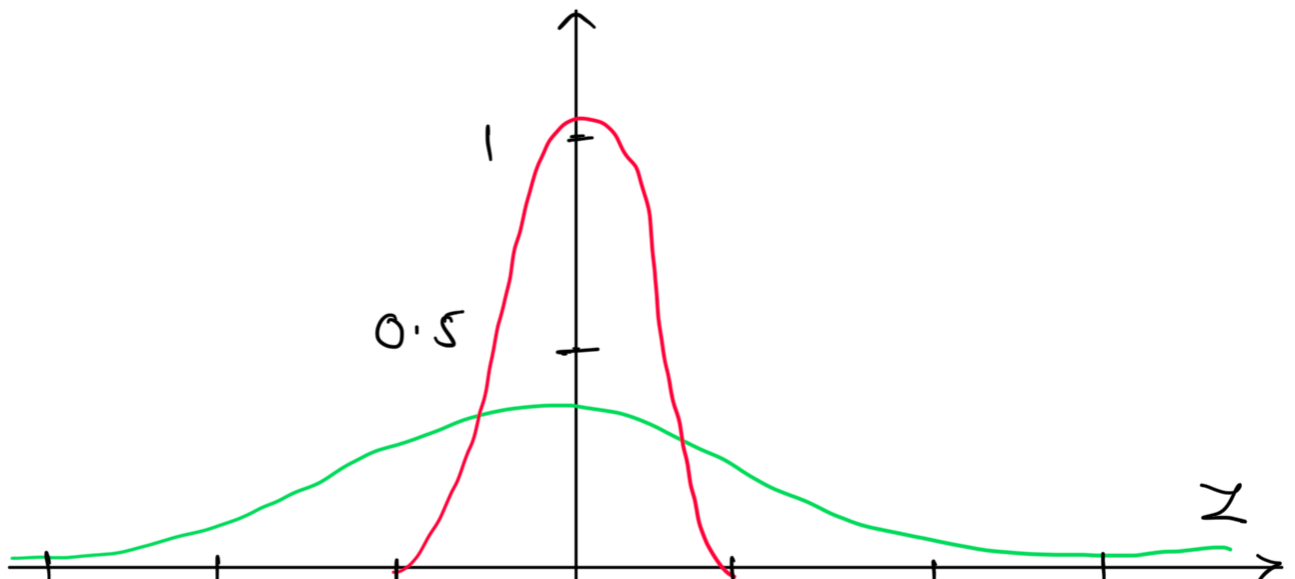




Entropy for Continuous variables

Area	Built in	Price
1200	1999	3.5
1800	2011	5.6
1400	2000	7.3
...	...	...

Area	Built in	Price
2200	1989	4.6
800	2018	6.5
1100	2005	12.8
...	...	...



-3 -2 -1 | 1 2 3

Which of the above datasets have higher entropy?

Whichever is less peaked.

Information Gain

It is a metric used to train Decision Trees. Specifically, this metric measures the quality of a split.

The information gain is based on the decrease in entropy after a data-set is split on an attribute. Constructing a decision tree is all about finding attribute that returns the highest information gain.

Information Gain = $E(\text{Parent})$

- $\{ \text{Weighted Average} \} * E\{ \text{children} \}$

Example

Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

Step 1 - Entropy of Parent

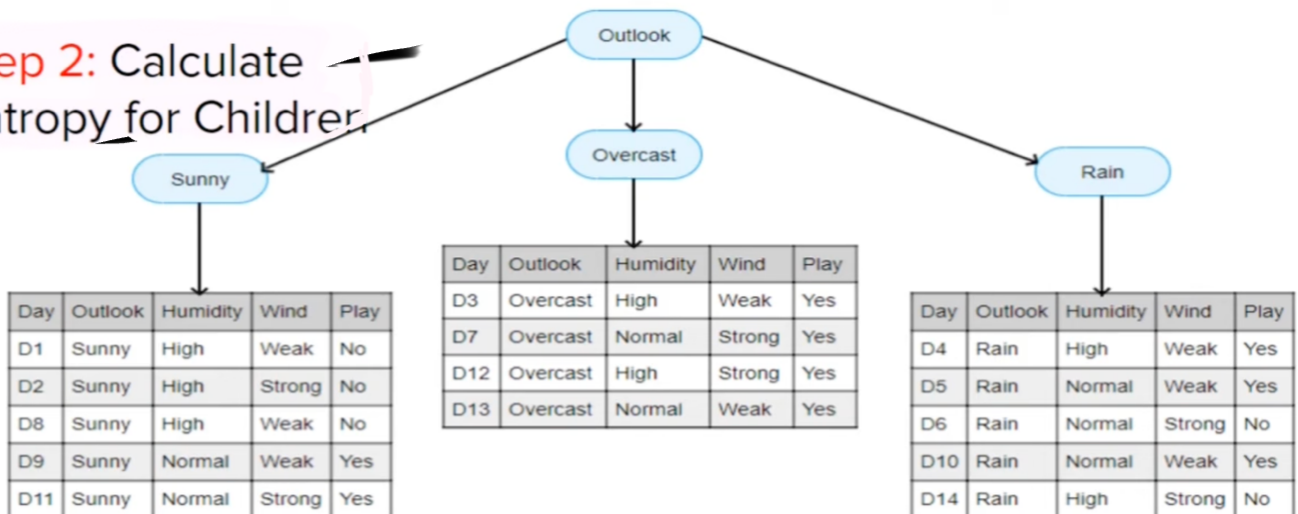
$$E(P) = -P_y \log_2(P_y) - P_n \log_2(P_n)$$

$$= \frac{9}{14} \log_2(9/14) - \frac{5}{14} \log_2(5/14)$$

$$E(P) = 0.94$$

Step 2: Calculate Entropy for children by splitting the data.

Step 2: Calculate Entropy for Children



$$E(S) =$$

$$-2 \log_2 2 -$$

$$E(O) =$$

$$-5 \log_2(5/5)$$

$$E(R) =$$

$$-3 \log_2(3/5)$$

$\frac{3}{5} \log_2 \frac{3}{5}$	$-\frac{0}{5} \log_2(0/5)$	$-\frac{2}{5} \log_2(2/5)$
$E(S) = 0.9$	$E(S) = 0$	$E(S) = 0.97$

Step 3: Calculate weighted Entropy of children

$$\text{Weighted Entropy} = \frac{5}{14} \times 0.97 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.97$$

$$WEC(\text{children}) = 0.69$$

PC(Overcast) is a leaf node as its entropy is 0.

Step 4: Calculate Information Gain

$$\begin{aligned}
 \text{Information Gain} &= E(\text{Parent}) - \{ \text{weighted avg} \} \times E(\text{children}) \\
 &= 0.97 - 0.69 \\
 IG &= 0.28
 \end{aligned}$$

So the information gain (or the decrease in entropy/impurity) when you split this data on the basis of outlook condition/column is 0.28

step 5: Calculate information gain for all the columns

Whichever column has the highest information Gain (maximum decrease in entropy) the algorithm will select that column to split the data.

Step 6 : Find information gain recursively

Decision tree then applies a recursive greedy search algorithm in top bottom fashion to find information Gain at every level of the tree.

Once a leaf node is reached (Entropy = 0), no more splitting is done.

Decision Trees Gini Impurity

Gini impurity a metric used by CART Classification and Regression Trees algorithms to measure the homogeneity of nodes in a decision tree, representing the probability of misclassifying a random element

It ranges from 0 (perfectly pure) to

0.5 (maximum impurity), with the goal of minimizing this value to create pure, well-separated nodes.

Formula

$$Gini = 1 - (P_y^2 + P_n^2)$$

OR

$$Gini = 1 - \sum_{i=1}^n (P_i)^2$$

Goal: Lower Gini Impurity indicates higher purity (more homogeneity)

Use in Decision Tree: It determines the best features to split nodes.

The split with the lowest weighted avg Gini impurity is chosen.

Example

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
12000	18	No

Salary	Age	Purchase
34000	31	No
15000	25	No
69000	57	Yes
25000	21	No
32000	28	No

$$\begin{aligned} G &= 1 - (P_y^2 + P_n^2) \\ &= 1 - (4/25 + 9/25) \\ G &= 0.48 \end{aligned} \quad \left| \quad \begin{aligned} G &= 1 - (P_y^2 + P_n^2) \\ &= 1 - (1/25 + 16/25) \\ G &= 0.32 \end{aligned}$$

Handling Numerical columns

S No	User Rating	Downloaded
1	3.5	Yes
2	4.6	Yes
3	2.2	No
4	1.6	Yes
5	4.1	No

6	3.9	No
7	3.2	No
9	2.9	Yes
10	4.8	Yes
11	3.3	No
12	2.5	Yes
13	1.9	Yes

Step 1 : Sort the data on the basis of numerical column.

S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

Step 2: Split the entire data on the basis of every value of user-rating

rating > 1.6

S No	User Rating	Downloaded
1	1.6	Yes

S No	User Rating	Downloaded
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

rating > 1.9

S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes

S No	User Rating	Downloaded
3	2.2	No
4	2.5	Yes
5	2.9	Yes
6	3.2	No
7	3.3	No

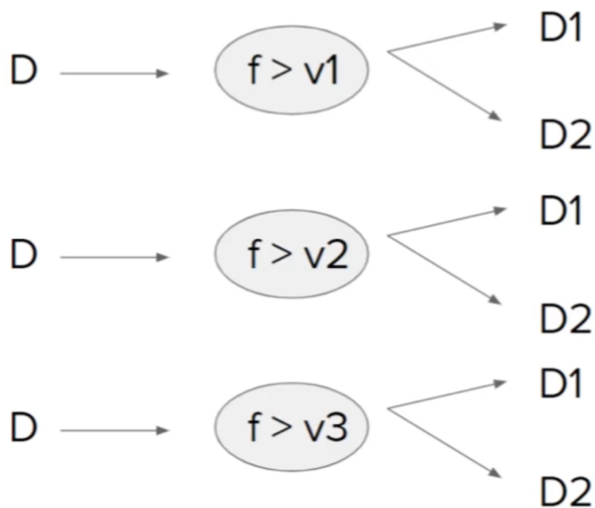
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

rating > 2.9

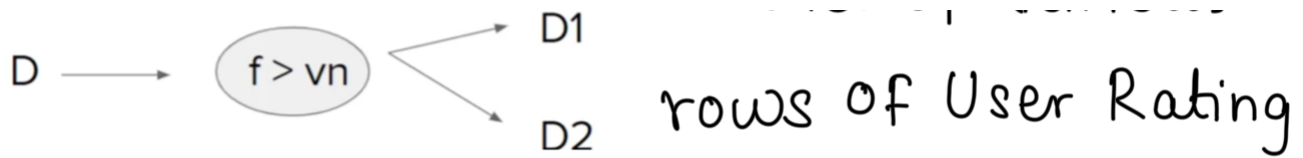
S No	User Rating	Downloaded
1	1.6	Yes
2	1.9	Yes
3	2.2	No
4	2.5	Yes
5	2.9	Yes

S No	User Rating	Downloaded
6	3.2	No
7	3.3	No
9	3.5	Yes
10	3.9	No
11	4.1	No
12	4.6	Yes
13	4.8	Yes

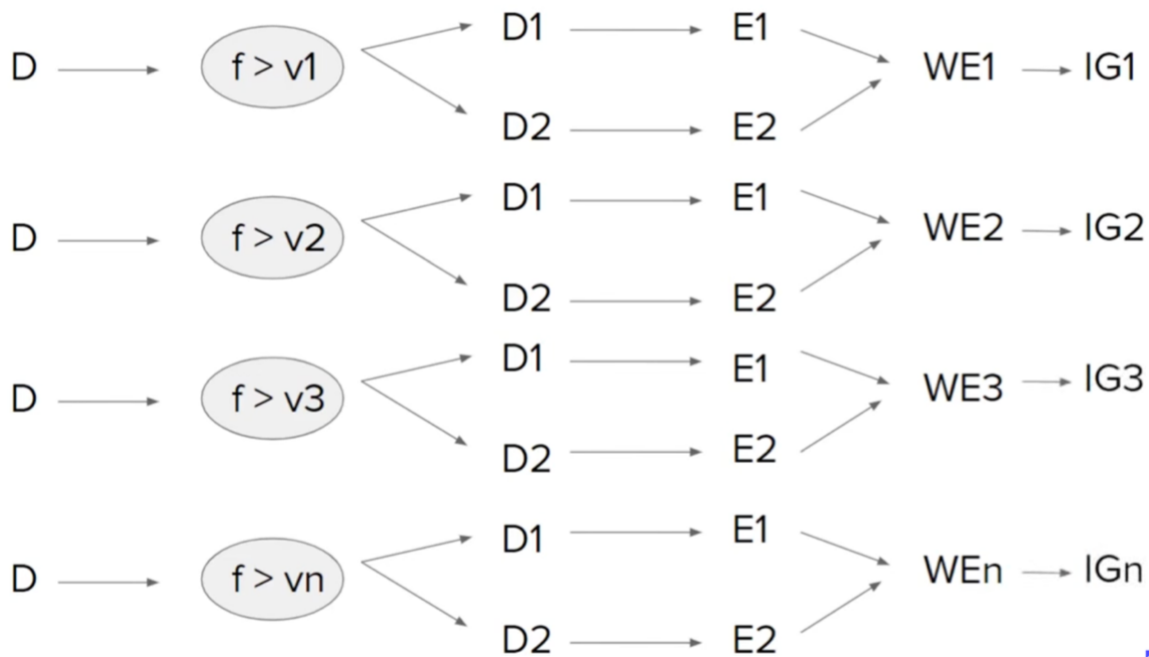
step 3



- Where D is the Dataset
- F is the column user rating
- $v_1, v_2 \dots v_n$ are the values of various

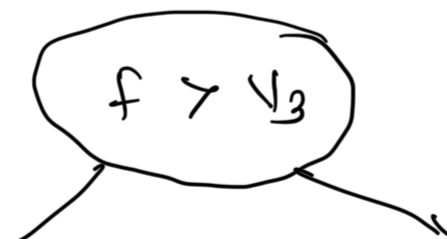


step 4



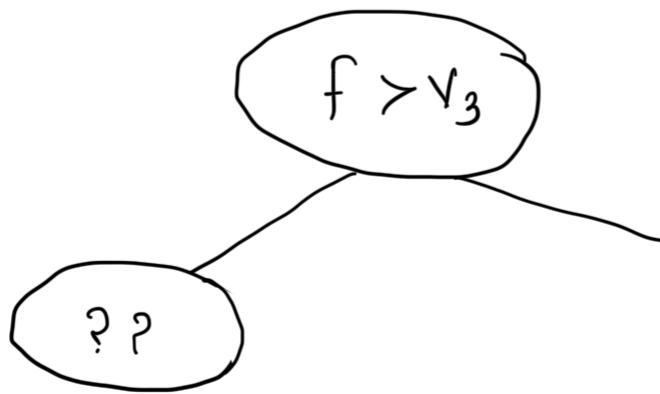
Step 5

$\max \{ IG_1, IG_2, IG_3, \dots, IG_n \}$
and set the splitting criteria



Step 6

Do this recursively until you get all the leaf nodes.



Even though entire thing looks computationally too expensive, this is the only right way of finding the splitting criteria.

This is done only once while training.